

Product Requirements Document (PRD)

ClassBridge AI

Product Name	ClassBridge AI
Document Type	Product Requirements Document
Version	2.0 (revised to match the current codebase)
Status	Working prototype — auth, logging, AI updates, and WhatsApp delivery are implemented and tested
Last Updated	September 2026

Note on this revision: Version 1.0 of this PRD described the product as originally envisioned. This version (2.0) has been rewritten against the actual `e:\ClassBridge` codebase (Next.js + FastAPI/SQLAlchemy/Neon Postgres) as of Phase 6. Every section reflects what is actually built, with explicit callouts wherever the shipped product diverges from the original plan.

1. Executive Summary

ClassBridge AI is a bilingual (now **trilingual**) communication platform that turns raw classroom data — attendance, grades, and teacher notes — into short, clear, empathetic updates that parents can actually understand and act on.

The platform’s defining principle is accessible, human-centered translation: AI converts academic jargon into plain-language, non-alarmist summaries delivered in **Simple English, Friendly Roman Urdu, and native Urdu script**, directly to a parent’s in-app feed and, optionally, WhatsApp — closing the communication gap regardless of a parent’s English fluency or education level.

One-line pitch: “Turning classroom data into parent understanding.”

What changed from v1.0: the original PRD scoped two output languages (Simple English + Roman Urdu). The shipped AI pipeline generates a third parallel version in real Urdu script (`urdu_text`), and each student now carries a `preferred_language` setting (`english` / `roman_urdu` / `urdu`). **CHANGED**

2. Problem Statement

- Millions of parents receive report cards and updates filled with academic jargon they cannot decode, leaving them unable to meaningfully support their child.
- Teachers juggling 30–40 students per class do not have time to write personalized, empathetic updates for every family.
- The result: parents stay in the dark about their child’s progress, and struggling students slip through the cracks until problems are severe.
- Existing school communication tools tend to be one-size-fits-all, English-only, and built around raw data dumps rather than parent-readable narratives.

(Unchanged from v1.0 — still the core problem the shipped product addresses.)

3. Goals & Objectives

Business / Project Goals

1. Demonstrate a working pipeline that converts structured classroom data into accurate, empathetic, trilingual parent-facing summaries. **DONE**
2. Prove that a non-alarmist, plain-language communication layer measurably increases parent understanding and engagement. Tone rules are enforced in the generation prompt (see §7.3), but no engagement metrics are collected yet (see §11).
3. Reduce the time teachers spend writing individual family updates. **DONE** — one-click generation with a review/edit step.

User Goals

- Teachers want to log attendance, grades, and notes quickly and trust that parents will receive an accurate, appropriately-toned update without extra writing effort. **DONE**
- Parents want to know — in a language and tone they understand — how their child is doing, and what (if anything) they should do about it. **DONE**
- ~~School admins want visibility into communication activity and which students are being flagged as at-risk across classes.~~ **NOT BUILT**

4. Non-Goals / Out of Scope (Current Version)

- Not a full learning management system (LMS) — does not replace gradebook or curriculum tools, only translates their output into parent communication.
- No two-way messaging or chat between parents and teachers in this version.

- No native mobile app — web-responsive dashboard for teachers, in-app feed + WhatsApp delivery for parents.
- No payment or premium-tier functionality.
- **No admin role.** **REMOVED** The original PRD specified a third “School Admin” persona with a cross-class oversight dashboard and account-management powers. This was never built: `profiles.role` is constrained at the database level to exactly `'teacher'` or `'parent'`, there is no admin router, no admin dashboard page, and no admin demo account. Signing up with `role: "admin"` is rejected.
- **No automated weak-subject flagging.** **REMOVED** The original §7.6 (“Early Weak-Subject Flagging”) and its `SubjectFlag` entity were never implemented — no trend-detection logic, no flag model, no surfaced alerts anywhere in the codebase.
- **No automated monthly progress reports.** **REMOVED** The original §7.5 and `ProgressReport` entity were never implemented as a scheduled job. A “Progress” page exists in both apps but currently renders static/demo data, not a real aggregation service.
- Real WhatsApp Business API integration is implemented (not merely “supported as configuration”), but ships with a mock provider by default so the demo works without live credentials.

5. Target Users & Personas

Persona	Description	Primary Need
Teacher	Teaches 30–40 students across multiple subjects; logs attendance, grades, and notes daily	A fast way to keep every parent informed without writing individual updates by hand
Parent (Roman/native Urdu-preferred)	Limited English fluency, checks WhatsApp daily, wants to know how her child is doing	Clear, jargon-free updates in Roman Urdu or Urdu script that tell her what to do next
Parent (English-fluent)	Comfortable with English reports but has little time to log into a school portal	Quick, digestible updates delivered where he already looks — WhatsApp/feed
School Admin	—	Removed — no admin persona exists in the shipped product (see §4). REMOVED

6. Key User Stories

1. As a teacher, I can log a student's attendance, grades, and a quick free-text note through a dashboard, so I don't have to write a separate update myself. **DONE**
2. As a teacher, I can review and edit an AI-generated summary before it's sent, so I retain control over what parents receive. **DONE** (draft → approved status)
3. As a teacher, I can also send one-off announcements (attendance alerts, school events, parent meetings, holiday notices, exam reminders, emergencies, general notices) targeted at all parents, a class, a section, or specific students. **NEW — not in v1.0**
4. As a parent, I receive a short, plain-language update in my preferred language (English, Roman Urdu, or Urdu script) on my feed. **DONE**
5. As a parent who opts in, I also receive the update on WhatsApp, and can see its delivery status. **DONE**
- ~~6. As a parent, I can see a monthly progress summary aggregating attendance, grades, and notes into one narrative.~~ **UI ONLY** ~~not backed by a real aggregation/scheduling service yet.~~
- ~~7. As a parent, I am notified early when my child is flagged as weak in a subject.~~ **NOT BUILT**
- ~~8. As a school admin, I can see an overview of communication activity...~~ **NOT BUILT — no admin role**
- ~~9. As a school admin, I can manage teacher and parent accounts...~~ **NOT BUILT — no admin role**

7. Functional Requirements

7.1 Authentication

- Email/password signup and login for **two** roles: teacher, parent. (*v1.0 said three, including admin — see §4.*)
- Session-based auth: an opaque random token in a `cb_session` httpOnly cookie, hashed (SHA-256) before storage in a `sessions` table; logout revokes server-side immediately.
- Passwords hashed with Argon2.
- Double-submit CSRF protection via a non-httpOnly `cb_csrf` cookie + `X-CSRF-Token` header on state-changing requests.
- Role-based access control enforced server-side via `require_teacher` / `require_parent` dependencies — never trusting a client-supplied role.
- Parents can only access data linked to their own child(ren), via a `parent_student_links` table (auto-linked by matching e-mail).
- Forgot/reset-password flow with expiring tokens.

7.2 Teacher Dashboard & Data Logging

- Teachers log attendance (present/absent/late, one record per student per day), grades per subject/assessment, and free-text notes per student.
- Teachers manage a student roster (create/edit/delete), with roll number, section, class, and parent contact info (email, WhatsApp number, home address, preferred language).
- Teachers can review and edit an AI-generated summary before it is sent to a parent.

7.3 Translation & Summary Engine

- Converts raw attendance, grade, and note data into a short, plain-language summary — no academic jargon.
- Generates **three** parallel versions per update: Simple English, Friendly Roman Urdu, and native Urdu script. Expanded from two in v1.0
- Enforces a non-alarmist tone via an explicit system prompt: never invents facts not given to it, never diagnoses conditions, never uses alarming words ("failing," "poor," "at risk," "behind"), never blames the student or parent.
- Provider-agnostic: a single `generate_bilingual_update()` entry point selects OpenAI, Gemini, or a deterministic mock provider purely from an `AI_PROVIDER` environment variable.
- The mock provider (default, and used by the test suite) only echoes back structured facts already on the update context — never fabricates content.

7.4 Delivery

- Summaries are delivered to the parent's in-app feed by default. DONE
- Parents who opt in receive the same summary via **real WhatsApp Business Cloud API delivery** (Meta Graph API), implemented behind a `WhatsAppProvider` abstraction (`mock` / `meta_cloud`).
- Delivery status (`pending` / `sent` / `delivered` / `read` / `failed`) is tracked per send attempt in an append-only `parent_update_deliveries` table, updated live via a signed WhatsApp webhook.
- **New in the shipped product:** teachers can also send **announcements** over the same WhatsApp pipeline, with delivery tracked per recipient in `announcement_recipients` . Publishing an announcement auto-sends WhatsApp immediately.

7.5 Monthly Progress Summaries

~~**Not implemented as specified in v1.0.** NOT BUILT No recurring/scheduled job aggregates a student's month into a `ProgressReport` row. A "Progress" page exists in both apps but currently reads from static demo data, not the real Postgres-backed data.~~

7.6 Early Weak-Subject Flagging

~~Not implemented:~~ **NOT BUILT** ~~No trend-detection logic, no `SubjectFlag` model or table, and no surfaced alerts anywhere.~~

7.7 Admin Oversight Dashboard

~~Not implemented — no admin role exists~~ **NOT BUILT** ~~(see §4). No cross-class visibility, no communication-activity dashboard, no account-management UI.~~

8. Primary User Flows

Teacher Flow

Log in → Select class/student → Log attendance, grade, and/or a quick note → Generate bilingual/trilingual draft → Teacher reviews/edits draft → Approve → Send → delivered to parent feed, and to WhatsApp if opted in.

Announcement Flow new — not in v1.0

Log in → Compose announcement (title, message, type, target audience) → Publish → recipients resolved from the real roster → delivered to feed + WhatsApp automatically.

Parent Flow

Log in → See children linked to the account → Read short update in preferred language on feed (or WhatsApp) → (Progress tab exists but shows demo content, not live aggregation yet.)

Admin Flow

~~Removed — no admin role in the shipped product.~~

9. System Architecture & Tech Stack

Layer	Technology (as shipped)	vs. v1.0
Frontend Framework	Next.js 14 (App Router), React 18, TypeScript	Matches
Styling	Tailwind CSS	Matches
Backend	FastAPI (sync throughout — one exception: the WhatsApp webhook, which needs <code>await request.body()</code> for raw-body HMAC signature verification)	Matches
Database / ORM	SQLAlchemy 2.0 + Alembic migrations, targeting Neon PostgreSQL in production	Matches; the “in-memory / lightweight store” assumption in v1.0 does not apply — a real Postgres DB has always

		been required (SQLite only for automated tests)
Authentication	Custom session-cookie auth (Argon2 + httpOnly cookie + CSRF)	More specific than v1.0's generic "FastAPI authentication"
AI / NLP Engine	Pluggable: OpenAI, Gemini, or mock, via <code>AI_PROVIDER</code> env var	Matches the provider-agnostic principle
Delivery Channel	In-app parent feed (Postgres-backed) + WhatsApp Business Cloud API (Meta Graph API)	Real working integration, not a placeholder
Languages Supported	Simple English, Friendly Roman Urdu, native Urdu script	Expanded from two to three
Deployment	Not yet configured in this repo (no Vercel/Firebase/Supabase config present)	v1.0's recommendation is aspirational, not yet acted on

Design principle (unchanged, and actually implemented): the Translation & Summary Engine and the WhatsApp delivery layer each call their external provider through an abstract interface, so switching providers is a configuration change, never a code change.

10. Data Model (Core Entities — as actually implemented)

Entity	Key Fields
User	id, email, password_hash (Argon2), is_active, email_verified_at
Profile	id, user_id, full_name, email, role (teacher parent only — no admin)
Session	id, user_id, session_token_hash, created_at, revoked_at
PasswordResetToken	id, user_id, token_hash, expires_at
Student	id, teacher_id, full_name, class_name, grade_level, section, roll_number, student_key, parent_name, parent_email, whatsapp_number, home_address, location, preferred_language (english/roman_urdu/urdu)
ParentStudentLink	links a parent User to a Student, auto-created by matching e-mail
AttendanceRecord	student_id, date, status (present/absent/late) — one row per student per day
GradeRecord	student_id, subject, score, max_score, assessment_name, assessment_date
TeacherNote	student_id, teacher_id, note text, created_at

ParentUpdate	student_id, teacher_id, english_text, roman_urdu_text, urdu_text (three versions), status (draft/approved/sent), ai_model, source_snapshot
ParentUpdateDelivery	one row per WhatsApp send attempt (append-only), parent_update_id, whatsapp_number_snapshot, provider_message_id, status
Announcement new	teacher_id, title, message, announcement_type, target_type, status
AnnouncementRecipient new	announcement_id, parent_user_id, student_id, delivery_status, provider_message_id
SubjectFlag	Does not exist — not implemented.
ProgressReport	Does not exist — not implemented.

11. Non-Functional Requirements

- **Security:** Role-based access control server-side only; a 404 (not 403) is returned when a resource exists but isn't the caller's, so responses never leak which IDs are real; Argon2 password hashing; httpOnly session cookies; CSRF double-submit protection.
- **Accuracy & Tone:** Enforced via an explicit system prompt rather than post-hoc review alone; teacher can still edit before sending.
- **Accessibility:** Now covers three written forms (English, Roman Urdu, native Urdu script), a step beyond v1.0's two-language scope.
- **Reliability:** Delivery status tracked per attempt (append-only); a retry after a failed send is allowed while a duplicate send after success is blocked.
- **Performance:** Not yet measured/benchmarked — no explicit SLA or load testing exists.
- **Engagement measurement:** Not implemented — no analytics/engagement-tracking hooks, despite being a stated business goal (see §15).

12. Success Metrics (for a prototype/demo context)

- End-to-end pipeline (data logging → AI summary → trilingual delivery) functions correctly — verified by an automated `pytest` suite covering auth, students, attendance, grades, notes, parent updates, WhatsApp delivery, and announcements.
- Delivery status (feed and WhatsApp) is tracked for all sends, including webhook-driven status updates from Meta.
- ~~Monthly progress summaries and weak-subject flags generate correctly from seeded data — not applicable, these features don't exist yet.~~

13. Assumptions & Constraints

- The backend requires a real PostgreSQL database (Neon, in this project) at all times outside the test suite — no in-memory/lightweight demo-store mode, unlike v1.0's assumption.
- AI summary generation depends on an available OpenAI or Gemini API key when `AI_PROVIDER` is set to a real provider; the mock provider lets the app run without one.
- Real WhatsApp delivery depends on Meta Cloud API credentials being configured; otherwise the mock WhatsApp provider is used. See `docs/whatsapp-setup.md` .
- Urdu-script phrasing quality depends on the underlying LLM; older rows created before this field existed have `urdu_text = NULL` and are never backfilled with a guess.

14. Risks & Mitigations

Risk	Mitigation
AI-generated summaries could sound alarmist despite simplification intent	Enforced tone rules baked directly into the generation system prompt; teacher reviews/edits before send
WhatsApp delivery depends on parents' phone numbers being correct and WhatsApp-enabled	In-app feed serves as a fallback channel; delivery status tracked per attempt and failures are visible
Teachers may not consistently log attendance/grades/notes	Lightweight, fast-entry dashboard UI (no automated nudges/reminders implemented yet)
Sensitive student data requires careful access control	Server-side role checks only; 404-not-403 on ownership mismatches to avoid ID enumeration
Dependence on third-party LLM (OpenAI/Gemini) or WhatsApp (Meta) availability	Both integrations sit behind a small abstract-provider interface with a mock implementation
No admin oversight new risk, not in v1.0	Currently unmitigated — would need a new role and access-control model designed from scratch

15. Future Roadmap (Not Yet Implemented)

- Automated monthly progress report generation, replacing the current static demo Progress page.
- Early weak-subject flagging (trend detection over grades/attendance/notes).
- A school-admin role: cross-class oversight dashboard, account management, communication-activity metrics.

- Parent engagement analytics (open rates, response patterns).
- Two-way messaging between parents and teachers.
- Additional language support beyond English / Roman Urdu / Urdu script.
- Native mobile app.
- Deeper LMS/SIS integrations for automatic data import.
- Deployment configuration (hosting for frontend, managed Postgres + secrets for backend) — not yet present in this repo.

16. Appendix

Demo Accounts (prototype/testing only — as actually seeded)

(Changed from v1.0 — the demo accounts described there don't exist in this codebase.)

Seeded via `python -m scripts.seed_demo_users` (real signup path, hashed passwords, no admin account since the role doesn't exist):

- Teacher — `sarah.teacher@classbridge.ai` / `demo1234`
- Parent — `parent.ali@classbridge.ai` / `demo1234`

Glossary

- **Non-alarmist summary:** A generated update phrased constructively and calmly, focused on next steps rather than framing a student's performance as a failure.
- **Provider-agnostic:** A design approach where the AI provider (OpenAI vs. Gemini vs. mock) or delivery provider (WhatsApp mock vs. Meta Cloud API) can be swapped via configuration without changing application code.
- ~~**Weak-subject flag:** Defined in v1.0 but not built — no such concept exists in the shipped product.~~